Midterm-Progress Report

Aryan Goel (Working Individually)

USC ID: 3626468016

PROJECT TITLE:
An Embedded Feature Engineering Pipeline Using DuckDB

---

1. Introduction and Motivation

Modern data-driven applications require efficient processing of large-scale datasets for analytical tasks such as feature engineering, aggregation, and exploratory analysis. Traditional relational database systems such as MySQL and PostgreSQL are primarily optimized for transactional workloads (OLTP), where operations focus on inserting, updating, and retrieving individual rows.

However, analytical workloads (OLAP) differ significantly, often involving large scans, aggregations, and transformations across many rows but only a subset of columns. In such scenarios, row-based storage and tuple-at-a-time execution can lead to inefficient data access and increased execution time.

DuckDB is an embedded analytical database system designed specifically for OLAP workloads. It leverages **columnar storage** and **vectorized execution** to efficiently process large datasets within a single-node environment (Raasveldt & Mühleisen, 2019).

In this project, we design and implement a **feature engineering pipeline** using DuckDB and analyze how its internal architecture affects performance. Through execution plan analysis and experimental evaluation, we connect database internals with real application behaviour.

---

2. Database Overview And Architecture

2.1: Overview Of DuckDB

DuckDB is an in-process analytical database system that runs within the host application, eliminating the need for a separate database server. This design allows tight integration with data science workflows and reduces overhead associated with client-server communication.

The system is optimized for analytical queries that involve:

- large table scans
- aggregations
- filtering operations
- feature computation

2.2: Architecture

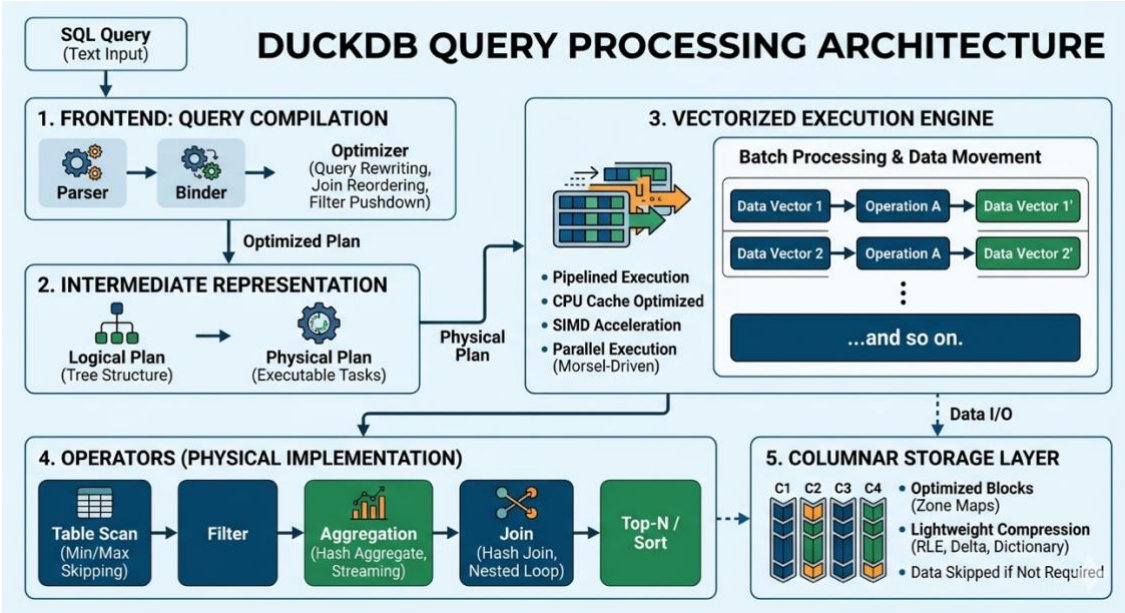


Fig 1. Underlying Architecture for DuckDB

DuckDB follows a layered architecture in which SQL queries are first parsed and optimized into a physical execution plan. This plan consists of operators such as sequential scans, filters, and aggregations, which are executed using a **vectorized execution engine**.

The execution engine processes data in batches (DataChunks) rather than row-by-row, improving CPU efficiency and reducing overhead. This design is inspired by earlier systems such as MonetDB/X100, which introduced vectorized query execution for analytical workloads (Boncz et al., 2005).

2.3: Columnar Storage

DuckDB stores data in a **column-oriented format**, where each column is stored separately in contiguous memory segments. This enables **column pruning**, allowing the system to read only the required attributes for a query.

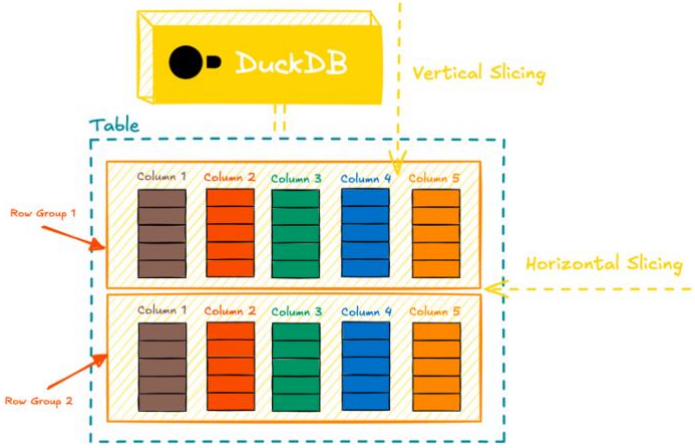


Fig 2. Columnar Slicing shown

In our project, the aggregation query:

```
SELECT user_id, SUM(amount)
FROM transactions
GROUP BY user_id;
```

only scans the `user_id` and `amount` columns, while ignoring other attributes such as `city`, `category`, and `payment_method`.

This behaviour demonstrates how columnar storage reduces unnecessary data access and improves query performance.

Fig 3. Physical plan showing column pruning where only required columns (user_id, amount, transaction_time) are scanned

And when the query was:

```
SELECT *
```

Then:

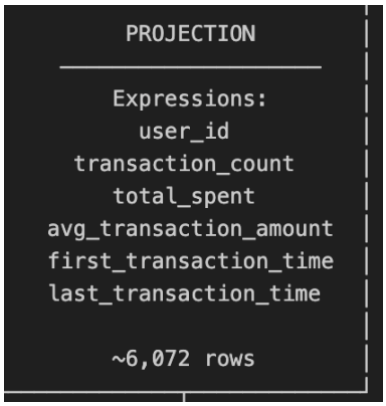


Fig 4. Physical plan showing full column scan when all attributes are requested

This behavior demonstrates how columnar storage reduces unnecessary data access and improves query performance.

2.4: Vectorized Execution

DuckDB uses a **vectorized execution model**, where operators process data in batches instead of individual tuples.

From the execution plan, we observe the operator pipeline:

```
SEQ_SCAN → HASH_GROUP_BY → TOP_N
```

- **SEQ_SCAN** reads data from storage
- **HASH_GROUP_BY** performs aggregation
- **TOP_N** selects the highest results efficiently

This pipeline reflects a push-based execution model where intermediate results are passed between operators in batches. This reduces function-call overhead and improves cache locality.

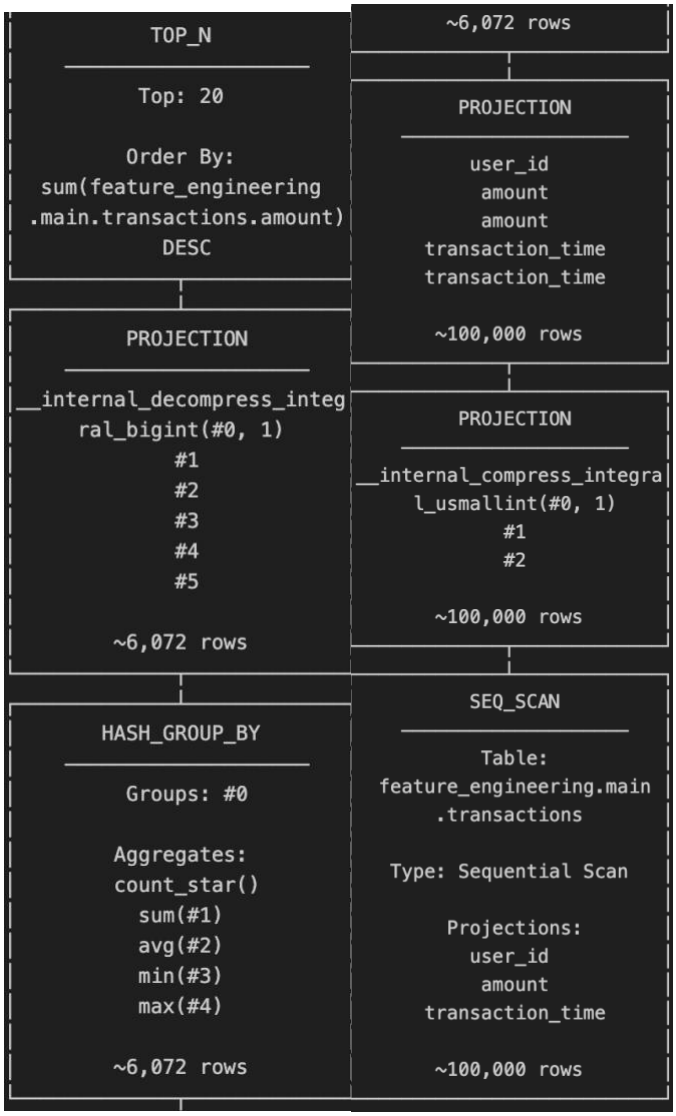


Figure 5. Execution pipeline showing SEQ_SCAN → HASH_GROUP_BY → TOP_N operators

---

3. Application Design

3.1 Overview

The application simulates a **feature engineering pipeline** over a transactional dataset.

Pipeline:

1. Generate synthetic transactional data

2. Load data into DuckDB
3. Execute feature engineering queries
4. Store results as a feature table
5. Visualize results using Streamlit

3.2 Feature Engineering

The system computes user-level features including:

- total transaction value
- transaction count
- average transaction amount
- fraud-related metrics
- category diversity
- temporal activity

These features are typical inputs for machine learning models, making the pipeline realistic.

3.3 Streamlit Interface:

The application includes an interactive Streamlit interface with:

- user-level transaction exploration
- feature query execution
- fraud analysis dashboard
- execution plan visualization

The UI allows users to interactively trigger queries and observe results, bridging the gap between database internals and application behaviour.

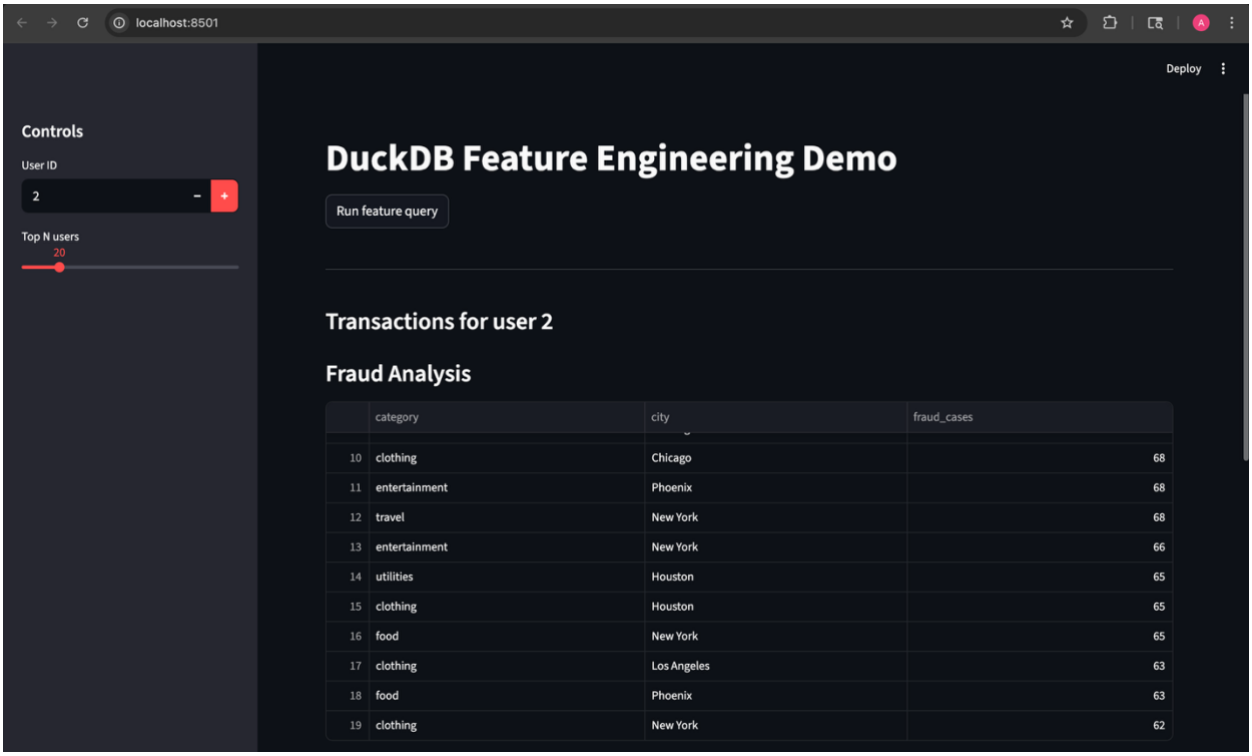


Figure 5: Streamlit interface for feature engineering pipeline

	transaction_id	user_id	merchant_id	amount	category	payment_method	city	is_fraud	transaction_time
0	60138	2	420	473.94	clothing	paypal	Houston	0	2023-12-19 04:53:00
1	75159	2	74	218.1	tech	credit_card	Los Angeles	0	2023-11-07 10:27:00
2	66020	2	412	465.83	tech	credit_card	New York	0	2023-10-26 05:03:00
3	90446	2	373	61.1	travel	credit_card	Chicago	0	2023-10-15 20:28:00
4	36979	2	122	332.87	travel	paypal	Chicago	0	2023-10-05 16:28:00
5	67667	2	223	6	clothing	credit_card	Houston	0	2023-08-03 00:14:00
6	94204	2	378	299.73	travel	google_pay	Phoenix	0	2023-07-22 07:10:00
7	90802	2	210	398.07	tech	google_pay	Houston	0	2023-07-06 12:12:00
8	35493	2	269	349.17	travel	apple_pay	Chicago	0	2023-06-18 07:08:00
9	98987	2	121	367.74	tech	credit_card	New York	0	2023-06-07 23:20:00

Run feature query

Feature query result

	user_id	transaction_count	total_spent	avg_transaction_amount	last_transaction_time
0	2416	34	9925.46	291.9253	2023-12-17 11:32:00
1	4840	36	9912.67	275.3519	2023-12-18 14:35:00
2	4483	35	9846.06	281.316	2023-12-28 22:05:00
3	4714	36	9653.03	268.1397	2023-12-21 08:51:00
4	3675	34	9601.37	282.3932	2023-12-19 19:27:00
5	3033	34	9600.8	282.3765	2023-12-31 05:11:00
6	3233	33	9553.25	289.4924	2023-12-18 11:51:00
7	4720	35	9462.56	270.3589	2023-12-28 03:37:00
8	676	34	9447.02	277.8535	2023-12-31 07:37:00
9	2424	34	9293.32	273.3329	2023-12-23 13:30:00

Figure 6: Feature engineering query results showing top users

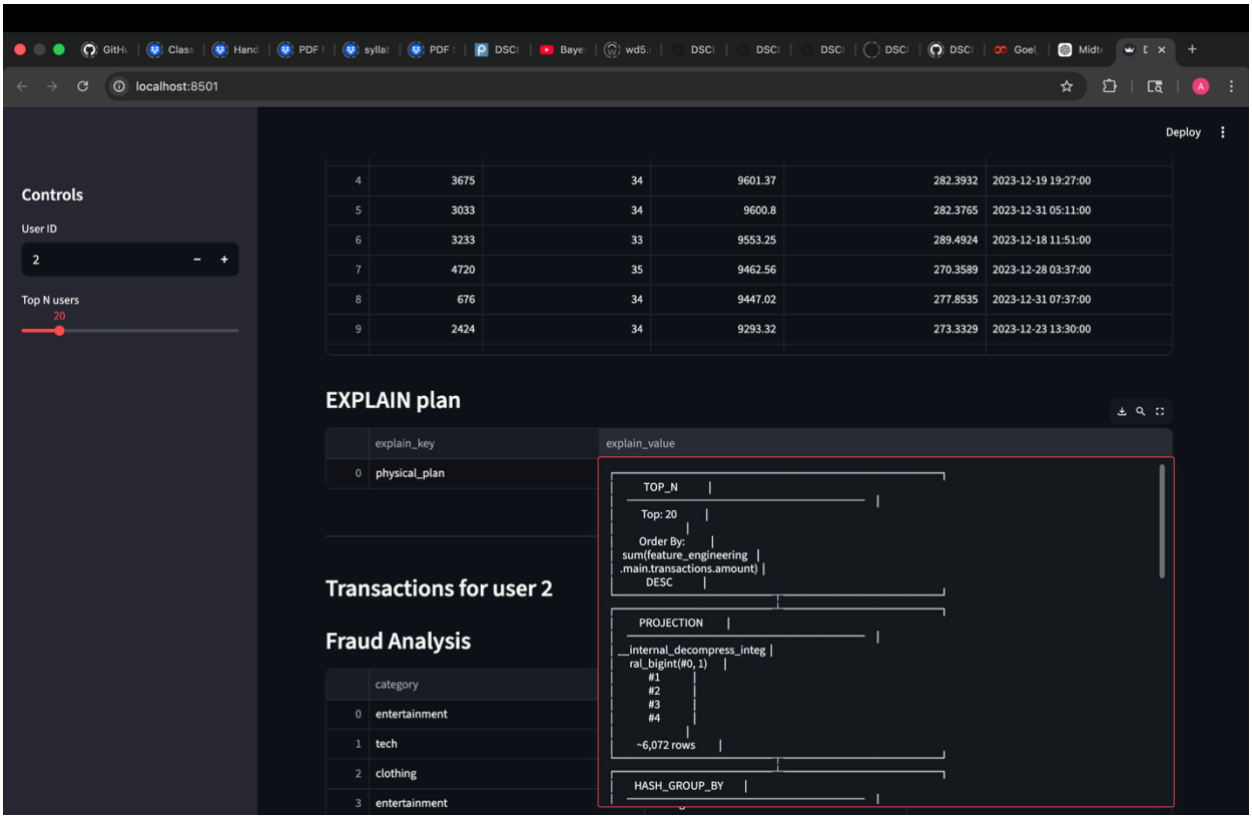
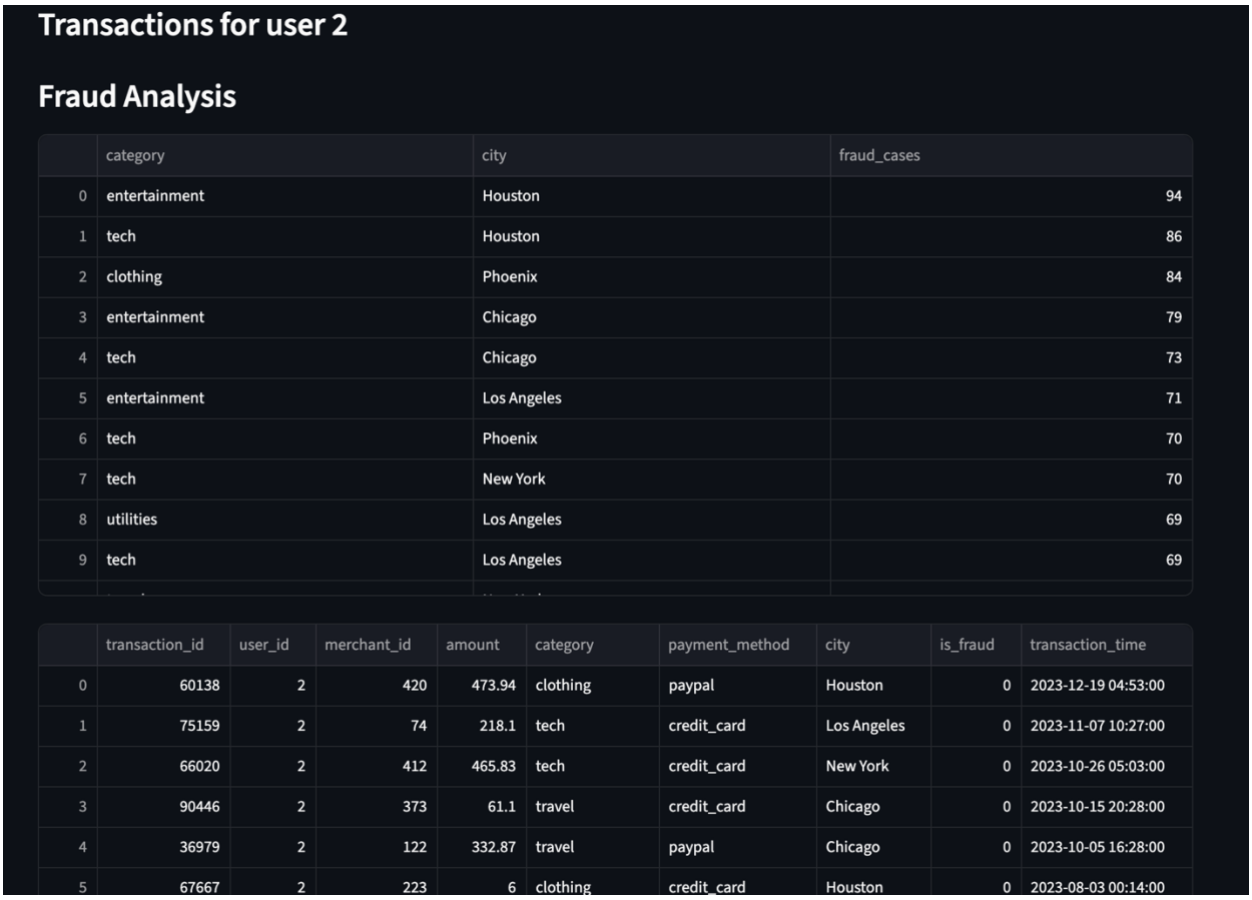


Figure 7: User-level transaction exploration



Transactions for user 2

Fraud Analysis

	category	city	fraud_cases
0	entertainment	Houston	94
1	tech	Houston	86
2	clothing	Phoenix	84
3	entertainment	Chicago	79
4	tech	Chicago	73
5	entertainment	Los Angeles	71
6	tech	Phoenix	70
7	tech	New York	70
8	utilities	Los Angeles	69
9	tech	Los Angeles	69

	transaction_id	user_id	merchant_id	amount	category	payment_method	city	is_fraud	transaction_time
0	60138	2	420	473.94	clothing	paypal	Houston	0	2023-12-19 04:53:00
1	75159	2	74	218.1	tech	credit_card	Los Angeles	0	2023-11-07 10:27:00
2	66020	2	412	465.83	tech	credit_card	New York	0	2023-10-26 05:03:00
3	90446	2	373	61.1	travel	credit_card	Chicago	0	2023-10-15 20:28:00
4	36979	2	122	332.87	travel	paypal	Chicago	0	2023-10-05 16:28:00
5	67667	2	223	6	clothing	credit_card	Houston	0	2023-08-03 00:14:00

Figure 8: Fraud analysis dashboard generated using DuckDB queries

4. Mapping Internals to Application Behavior

In this whole Section, I have explained how I used some queries to map for application behavior plus some screenshots for the output, I have already mentioned and showed in the earlier section. For some parts I have mentioned my GitHub repo regarding this project in the reference section to see the whole project. There under the Outputs folder, you may find the explain_user_feature.txt giving a tree type format whose Screenshots I have pasted above also, gives you the full picture what is happening behind the back of DuckDB.

4.1 Aggregation Query (Shown in Fig 3):

```
SELECT user_id, SUM(amount)
FROM transactions
GROUP BY user_id;
```

Execution Behavior:

- Sequential scan reads only required columns
- Hash aggregation groups data by `user_id`
- Results are projected to output

Insight:

This demonstrates:

- column pruning
- vectorized aggregation

4.3 Full Table Scan (Shown in Fig 4):

```
SELECT * FROM transactions;
```

Observation:

- All columns are scanned
- Execution time is significantly higher (~0.095s)

Insight:

Wide queries increase data access cost in columnar systems.

4.4 Selective Filtering:

```
SELECT * FROM transactions WHERE is_fraud = 1;
```

Observation:

- Uses sequential scan with filter
- No index usage

Insight:

DuckDB prefers **scan + filter** over index-based lookup, which is suitable for analytical workloads.

4.5 Multi-column Aggregation

```
SELECT category, city, SUM(amount)
FROM transactions
GROUP BY category, city;
```

Observation:

- Uses hash-based aggregation
- Only required columns are scanned

---

5. Experiments

outputs > benchmark_results.csv > data

```
query_name,avg_seconds,row_count
narrow_aggregation,0.002329,5000
wide_scan,0.083707,100000
selective_filter,0.002906,1993
multi_column_groupby,0.001849,30
point_lookup,0.00086,18
```

The results highlight key advantages of DuckDB:

- Narrow queries benefit from column pruning
- Aggregation queries execute efficiently
- Wide queries incur higher cost
- Filtering remains efficient despite full scans

---

6. Validation of System Design:

The experimental results align with the design principles described in the DuckDB paper.

Paper Concept	Observation
Columnar storage	Only required columns scanned
Vectorized execution	Fast aggregation
Pipeline execution	SEQ_SCAN → HASH_GROUP_BY
Scan-based model	No index usage

This confirms that DuckDB's architecture is well-suited for analytical workloads.

---

7. Comparison with Traditional Databases

Feature	MySQL/PostgreSQL	DuckDB
Storage	Row-based	Columnar
Execution	Tuple-at-a-time	Vectorized
Optimization	Index-based	Scan-based
Workload	OLTP	OLAP

8. Progress & Challenges

Progress

- Implemented dataset generation
- Built DuckDB pipeline
- Developed feature queries
- Created Streamlit interface
- Conducted experiments
- Analyzed execution plans

Challenges

- Understanding execution plans
- Mapping operators to internals
- Interpreting vectorized execution
- Designing meaningful experiments

---

9. References:

[1] Raasveldt, M., & Mühleisen, H. (2019).
DuckDB: an embeddable analytical database. CIDR.

[2] Boncz, P., Zukowski, M., & Nes, N. (2005).
MonetDB/X100: Hyper-Pipelining Query Execution.

[3] DuckDB Documentation
https://duckdb.org/docs/stable/internals/

[4] DuckDB YouTube Channel:
https://www.youtube.com/%40duckdb/videos?utm_source=chatgpt.com

[5]: DuckDB Internals with CTO Mark Raasveldt

[6] My GitHub repo for the project:
https://github.com/goelaryan001/551_FinalProject_DuckDB_FeatureEngineering.git